

ConsensusCode-Evolve: Token-Efficient Generative Code Search for Consensus-Aware Multi-Agent Inventory Automation

Author Name

Abstract—Generative AI is creating new opportunities for industrial automation, but direct deployment of large language models (LLMs) as online controllers remains difficult because of cost, safety, interpretability, and reproducibility concerns. This paper proposes *ConsensusCode-Evolve*, a simulator-grounded framework that uses an LLM as an offline code-evolution engine for multi-agent inventory automation. The target problem is multi-site replenishment and transshipment, a representative discrete manufacturing and supply-operation task in which local inventory agents must coordinate material movements under demand uncertainty, lead times, capacity limits, and asymmetric cost structures. Instead of asking an LLM to control the system online, the proposed method asks it to generate constrained Python policy functions for replenishment, transshipment proposal, and consensus gating. Candidate policies are validated by an abstract-syntax-tree safety checker, evaluated in a physical inventory simulator, and selected through an archive-based loop using compact numerical feedback. A consensus-seeking layer models donor willingness, receiver urgency, and fairness debt, so transshipment actions are accepted only when they satisfy both local and system-level requirements. The resulting policies are lightweight, executable without an LLM at deployment time, and directly inspectable by engineers. The paper outlines the method, safety constraints, evaluation protocol, and planned comparisons against genetic programming, heuristic, and learning-based baselines. The central hypothesis is that simulator-grounded generative code search can discover interpretable, consensus-aware inventory policies with lower token cost than online LLM agents while retaining automation-oriented robustness and deployability.

Index Terms—Generative AI, industrial automation, large language models, multi-agent systems, consensus seeking, inventory control, transshipment, code generation, simulation optimization.

I. INTRODUCTION

Industrial automation increasingly requires decision-making systems that can coordinate distributed assets, adapt to uncertain operating conditions, and remain interpretable to engineers. In discrete manufacturing and supply operations, material-flow decisions such as replenishment, inventory allocation, and transshipment are not merely numerical optimization tasks. They involve multiple sites or production units with local constraints, asymmetric costs, and sometimes conflicting interests. A site with surplus inventory may be asked to support another site, but such a transfer can be harmful if the donor later faces a demand surge. This makes multi-site inventory

automation a natural setting for studying collaborative industrial task agents and consensus-aware control.

The special issue theme on generative AI (GAI) for industrial automation is especially relevant to this problem. GAI methods, industrial large models, and industrial task agents can potentially support production-line modeling, adaptive scheduling, multi-objective optimization, and cooperative decision-making. However, industrial automation imposes requirements that differ from open-ended language tasks. Generated decisions must respect physical mechanisms, capacity limits, lead times, safety constraints, and deployment budgets. Moreover, many industrial systems cannot afford to call a large model at every control step. A useful GAI method for automation should therefore be grounded in physical simulators, lightweight at deployment time, and transparent enough to support engineering inspection.

Recent progress in LLM-based code generation and program search suggests one promising path. AlphaCode showed that large language models can generate many candidate programs and use selection mechanisms to solve competitive programming problems [1]. CodeRL incorporated execution feedback into code generation [2]. Self-Refine and Reflexion demonstrated that language models can iteratively improve outputs using compact feedback or verbal memory [3], [4]. Voyager used LLM-generated executable code as reusable skills in an embodied environment [5], and Eureka showed that LLMs can write reward functions evaluated through simulation [6]. FunSearch is particularly relevant because it combines LLM-generated programs with automatic evaluation to discover high-performing mathematical constructions [7]. These studies suggest that an LLM need not be the controller itself; instead, it can serve as a generator of candidate executable programs, while an external evaluator provides grounding.

This paper transfers that idea to industrial inventory automation. We propose *ConsensusCode-Evolve*, a token-efficient framework in which an LLM evolves small, safe Python policy functions offline. Each candidate policy contains three functions: a replenishment policy, a transshipment proposal policy, and a consensus gate. The replenishment function determines local order quantities. The transshipment proposal function suggests pairwise material movements. The consensus gate accepts, scales, or rejects proposed transfers based on donor willingness, receiver urgency, network benefit, and fairness debt. Candidate code is statically checked before execution and then evaluated in a multi-site inventory simulator. Only compact performance summaries and selected policy code

snippets are returned to the LLM for the next mutation round.

The proposed approach differs from direct LLM planning in three ways. First, the LLM is used offline, which reduces token cost and avoids dependence on a large model during deployment. Second, generated code is constrained to a small safe subset of Python, making the final policies inspectable and reproducible. Third, the method embeds autonomous multi-agent consensus seeking into the action layer, so transshipment decisions are not treated as forced centralized commands but as negotiated actions among local inventory agents.

The intended contributions are as follows:

- A simulator-grounded generative code-evolution framework for multi-site replenishment and transshipment automation.
- A consensus-aware policy representation that separates replenishment, transshipment proposal, and agreement gating.
- A lightweight safety mechanism based on abstract syntax tree validation for generated industrial policy code.
- An evaluation protocol comparing LLM-evolved policies with genetic programming, heuristic, and learning-based baselines under cost, service, fairness, and robustness metrics.

This draft does not claim empirical superiority yet. The present implementation establishes the framework and an initial prototype. Full experimental validation should compare multiple scenarios, seeds, and token budgets before making performance claims.

II. METHOD

A. Problem Setting

We consider a multi-site inventory system with N locations. At each period t , site i has inventory level $I_{i,t}$, pipeline inventory $P_{i,t}$, capacity C_i , holding cost h_i , lost-sales cost b_i , fixed and per-unit ordering costs, and short-horizon demand forecasts. The automation task is to choose replenishment quantities and transshipment flows. Replenishment orders arrive after a lead time, while transshipment immediately redistributes inventory between sites subject to availability and transfer cost.

The objective is to minimize long-run average cost,

$$J = \frac{1}{T} \sum_{t=1}^T \left(C_t^{\text{trans}} + C_t^{\text{hold/lost}} + C_t^{\text{order}} \right), \quad (1)$$

while maintaining service quality and avoiding unstable or unfair transfer patterns. In contrast to a purely centralized view, we treat each site as an autonomous inventory agent with local interests. A transshipment is desirable only if it benefits the receiver without exposing the donor to excessive future risk.

B. Policy-Code Representation

ConsensusCode-Evolve represents an inventory policy as three constrained Python functions:

- `replenishment_policy(site_state, global_state)` returns a nonnegative order quantity for one site.

- `transshipment_proposal(source_state, target_state, global_state)` returns a signed or directional proposed transfer quantity.
- `consensus_gate(source_state, target_state, proposed_quantity, history)` returns the accepted transfer quantity after agreement filtering.

The policy input exposes physically meaningful variables, including inventory, pipeline, capacity, holding cost, lost-sales cost, short forecasts, total network inventory, total forecast, inventory imbalance, and historical transfer imbalance. This design gives the LLM enough structure to generate useful policies without seeing long trajectories.

C. Consensus-Seeking Transshipment

For each potential pairwise transfer, the framework first computes a proposed quantity and then applies a consensus gate. The gate can be interpreted as an agreement mechanism among local agents. A typical form is

$$q_{ij,t}^{\text{acc}} = q_{ij,t}^{\text{prop}} \cdot w_{i,t}^{\text{donor}} \cdot u_{j,t}^{\text{receiver}} \cdot f_{ij,t}^{\text{fair}}, \quad (2)$$

where $q_{ij,t}^{\text{prop}}$ is the proposed transfer from site i to site j , $w_{i,t}^{\text{donor}}$ measures donor willingness, $u_{j,t}^{\text{receiver}}$ measures receiver urgency, and $f_{ij,t}^{\text{fair}}$ penalizes repeated one-way support. These terms may be hand-designed in seed policies or generated directly by the LLM as executable code.

This structure is important for industrial automation because it prevents the GAI system from generating opaque transfer commands. Each accepted movement can be explained in terms of donor safety, receiver need, and fairness history.

D. Safe Generative Code Search

The LLM is used only between simulation batches. In each round, the algorithm provides a compact prompt containing the best policy code snippets and a table of metrics such as total cost, transshipment cost, holding/lost-sales cost, order cost, stockout proxy count, accepted transfer count, rejected transfer count, and fairness debt. The LLM then proposes new candidate policy files.

Before simulation, each candidate is checked by an abstract-syntax-tree validator. The current safe subset disallows imports, file operations, network access, random calls, classes, attributes, and unbounded external dependencies. It permits arithmetic, comparisons, simple conditionals, dictionary-state indexing, and calls to `abs`, `min`, `max`, and `round`. Candidates that fail validation are rejected without execution.

E. Evolution Loop

The search loop is summarized as follows:

- 1) Initialize an archive with human-written seed policies.
- 2) Evaluate each candidate policy in the inventory simulator across selected scenarios and seeds.
- 3) Store policy source code and compact metrics in an archive.
- 4) Select the top-performing and behaviorally distinct policies.

- 5) Prompt the LLM to generate a small number of mutations under strict API and safety rules.
- 6) Validate generated code, evaluate valid candidates, and update the archive.

This loop follows the spirit of program search with automatic evaluation while being adapted to industrial automation. The evaluator, rather than the LLM, determines whether a candidate is useful.

F. Evaluation Plan

The planned evaluation compares ConsensusCode-Evolve against: (i) the existing multi-tree genetic programming method for replenishment and transshipment, (ii) hand-designed consensus policies, (iii) LLM-evolved policies without the consensus gate, and (iv) classical inventory baselines such as (s, S) policies where applicable. The primary metrics are total average cost, holding/lost-sales cost, order cost, transshipment cost, stockout frequency, transshipment frequency, fairness debt, and robustness under stress-test scenarios.

The ablation study should vary the token budget, number of candidates per LLM call, consensus components, and archive-selection strategy. A useful automation-oriented result would show not only low cost, but also low token usage, stable simulator feasibility, interpretable policies, and improved coordination behavior compared with direct pairwise transshipment rules.

III. PLANNED EXPERIMENTS AND DISCUSSION

The initial prototype has implemented the safe policy API, AST validation, simulator adapter, seed policy evaluation, prompt generation, and generated-candidate evaluation. The next stage is to perform systematic experiments across two-site and three-site scenarios, multiple random seeds, and different holding/lost-sales asymmetries. Results should be reported conservatively. Until full experiments are complete, the central claim should be framed as a methodological contribution and a testable hypothesis rather than an established empirical improvement.

REFERENCES

- [1] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, T. Hubert, P. Choy, C. de Masson d’Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Goyal, A. Cherepanov, J. Molloy, D. Mankowitz, E. Sutherland Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals, “Competition-level code generation with AlphaCode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [2] H. Le, Y. Wang, A. D. Gotmare, S. Savarese, and S. C. H. Hoi, “CodeRL: Mastering code generation through pretrained models and deep reinforcement learning,” in *Advances in Neural Information Processing Systems*, 2022.
- [3] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, S. Welleck, B. P. Majumder, S. Gupta, A. Yazdanbakhsh, and P. Clark, “Self-refine: Iterative refinement with self-feedback,” in *Advances in Neural Information Processing Systems*, 2023.
- [4] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, 2023.
- [5] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *arXiv preprint arXiv:2305.16291*, 2023.
- [6] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, “Eureka: Human-level reward design via coding large language models,” in *International Conference on Learning Representations*, 2024.
- [7] B. Romera-Paredes, M. Barekatin, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, “Mathematical discoveries from program search with large language models,” *Nature*, vol. 625, pp. 468–475, 2024.